

Recurrent Neural Network | Forward Propagation | Architecture

➤ What is an RNN?

A **Recurrent Neural Network (RNN)** is a neural network designed to handle **sequential data**, where **previous information** is important for the current step.

Unlike normal neural networks (ANNs) that process all inputs **independently**, RNNs have **memory** they remember what happened in the past.

This makes them powerful for:

- Text (sentences, paragraphs)
 - Speech or audio
 - Stock prices
 - Time series, etc.
-

➤ Why RNNs?

Normal ANNs:

- Take **fixed-length input** (e.g., 784 pixels for MNIST)
- Treat each input as **independent**
- Can't remember earlier data

But **real-world data** often comes in sequences:

Example	Sequence Type	Why Memory Matters
Speech	Words over time	Each word depends on previous words
Stocks	Prices over days	Future depends on past
Music	Notes over time	Rhythm and melody depend on sequence

So, we need a network that:

- Handles **variable-length input**
- **Remembers** previous steps
- Uses **context** to predict future steps

That's why **RNNs were invented**.

➤ What Kind of Data Goes Into an RNN?

RNNs take **sequential input**, for example:

Example 1: Sentence

"I love deep learning"

Each word can be represented as a vector:

$x_1 = \text{I}$, $x_2 = \text{love}$, $x_3 = \text{deep}$, $x_4 = \text{learning}$

Example 2: Stock Prices

$x_1 = 100$, $x_2 = 102$, $x_3 = 101$, $x_4 = 104$

So, an RNN processes the sequence:

$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$

and tries to predict either:

- The **next value** (for forecasting), or
- A **label** for the whole sequence (like sentiment classification).

➤ Architecture of a Basic RNN

An RNN processes **one element at a time**, but **remembers what it saw before**.

It does this through a **loop** inside the network.

Components

1. **Input (x_t)** – the data at time step t
2. **Hidden State (h_t)** – memory of what's been learned so far
3. **Output (y_t)** – prediction at time step t
4. **Weights:**
 - ❖ W_{xh} (input $\rightarrow$ hidden)
 - ❖ W_{hh} (hidden $\rightarrow$ hidden)
 - ❖ W_{hy} (hidden $\rightarrow$ output)

➤ Flow of Data

At each time step t :

1. The network receives input x_t
2. It combines it with the **previous hidden state** h_{t-1}
3. It generates a **new hidden state** h_t
4. That hidden state is used to:
 - ❖ Predict the **output** y_t
 - ❖ Pass **memory** to the next step

➤ Mathematical Formulas

At time step t :

1. **Hidden state update:**

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

2. Output prediction:

$$y_t = \text{softmax}(W_{hy} h_t + b_y)$$

Explanation:

- W_{xh} : weights connecting input to hidden layer
 - W_{hh} : weights connecting previous hidden state to current
 - $\tanh$: activation function (helps keep outputs between -1 and 1)
 - softmax : used for classification (converts scores into probabilities)
-

➤ Intuition

Think of h_t as the **memory box**:

- At each step, it gets a bit of new information (x_t)
- Mixes it with old memory (h_{t-1})
- Updates its “understanding” of the sequence so far

So over time, the RNN **builds context**.

➤ Example (Step-by-Step Forward Propagation)

Imagine a sentence:

“I love AI”

Let’s say:

- $x_1 = \text{"I"}$
- $x_2 = \text{"love"}$
- $x_3 = \text{"AI"}$

Step 1:

- Input: x_1
- Previous memory: $h_0 = 0$
- Compute:

$$h_1 = \tanh(W_{xh} x_1 + W_{hh} h_0 + b_h)$$
- Output: $y_1 = \text{softmax}(W_{hy} h_1 + b_y)$

Step 2:

- Input: x_2
- Memory from last step: h_1

- Compute:

$$h_2 = \tanh(W_{xh} x_2 + W_{hh} h_1 + b_h)$$
- Output: $y_2 = \text{softmax}(W_{hy} h_2 + b_y)$

Step 3:

- Input: x_3
- Memory from last step: h_2
- Compute:

$$h_3 = \tanh(W_{xh} x_3 + W_{hh} h_2 + b_h)$$
- Output: $y_3 = \text{softmax}(W_{hy} h_3 + b_y)$

This process = **Forward Propagation in RNN**

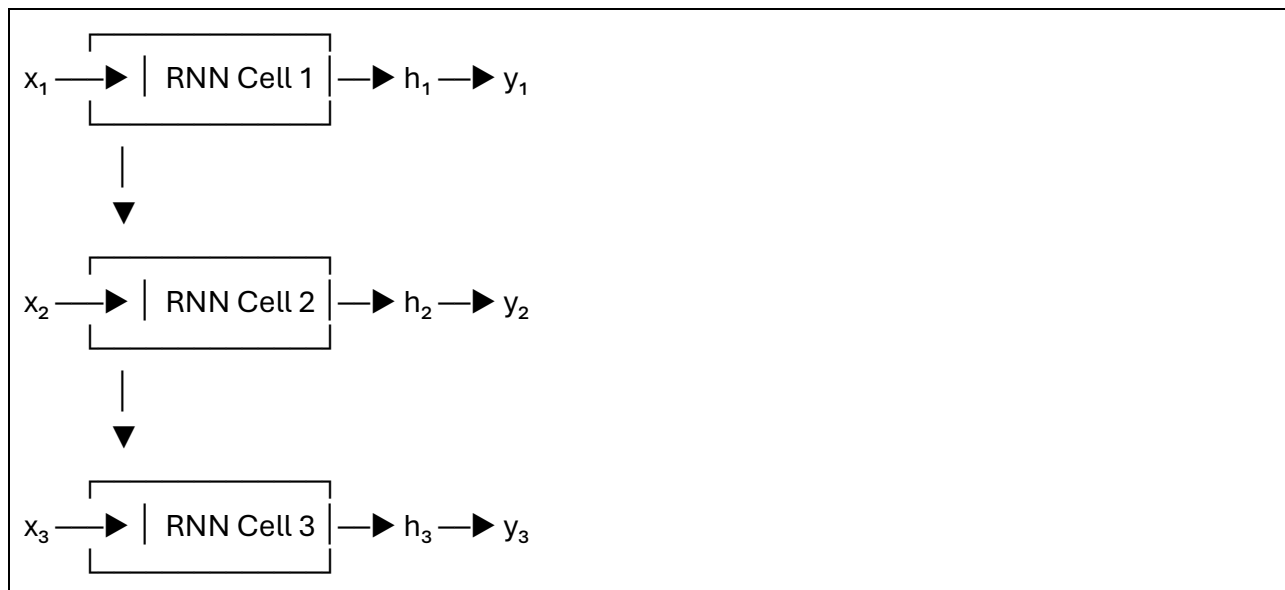
➤ What Happens in Forward Propagation

- RNN **processes the input sequence one step at a time**.
- Each step **updates the hidden state** using both new input and past memory.
- This hidden state is **carried forward**, allowing the network to **remember**.

This is what gives RNNs their **temporal behavior** — they understand patterns **across time**.

➤ Simplified Representation

Visually, we can unroll the RNN like this:



Each cell:

- Takes input (x_t)
- Uses memory (h_{t-1})

- Produces output (y_t)
- Passes new memory (h_t) to next cell

All cells share **the same parameters** (W_{xh} , W_{hh} , W_{hy}) → this makes training efficient.

➤ Code Example (Keras)

A simple RNN model in Keras:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([
    SimpleRNN(50, input_shape=(timesteps, features), activation='tanh'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy')
```

Explanation:

- SimpleRNN(50) → RNN layer with 50 hidden units
- input_shape=(timesteps, features) → how many time steps and features per step
- Dense(1) → output layer
- tanh → used for hidden state
- sigmoid → used for output (binary classification)

➤ Summary

Concept	Description
RNN	Neural network for sequential data
Memory	Keeps past information using hidden state
Forward Propagation	Processes one time step at a time, updating memory
Equation	$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$
Why Needed	Captures temporal patterns (e.g., speech, text, time series)
Main Components	Input (x_t), Hidden state (h_t), Output (y_t)
Code	Implemented using SimpleRNN in Keras